

Algorithmique et programmation impérative

Chapitre 7 : Listes

Hadrien Mélot
Service d'Algorithmique
Département d'Informatique
Faculté des Sciences



Chapitre 7. Listes

- 1 Une liste est une séquence d'éléments
- 2 Listes et opérateurs
- 3 Méthodes sur les listes
- 4 Objets et valeurs
- 5 Arguments de la ligne de commande (parenthèse)
- 6 Erreurs fréquentes et debug
- 7 Exercices interactifs
- 8 Pour aller plus loin (à lire par soi-même)
- 9 Glossaire

Illustration des listes

Illustration en auditoire :

- un peu de poésie
- énigme de la traversée (deux niveaux)

Les codes seront dévoilés plus tard.

Une liste est une séquence d'éléments

Une liste est une séquence d'éléments

Comme une chaîne de caractères, une **liste** est une séquence de valeurs, mais :

- dans une chaîne les valeurs sont des caractères, dans une liste elles peuvent être de n'importe quel type
- une chaîne est une séquence immuable, une liste est une séquence que l'on peut modifier
- une liste est définie en spécifiant ses éléments, séparés par des virgules, entre crochets (par ex. : `[1, 2, 3]`)

Création de listes

Remarques :

- une liste peut contenir des entiers, des chaînes, des tuples, des listes ou n'importe quelle autre type de valeur
- la fonction `len` peut être appliquée sur une liste
- une liste peut être vide

```
>>> numbers = [1, 2, 4, 7]
>>> print(numbers)
[1, 2, 4, 7]
>>> len(numbers)
4
>>> mots = ["bonjour", "bonsoir", "salut", "hello"]
>>> empty = []
>>> type(empty)
<class 'list'>
>>> len(empty)
0
```

Création de listes

Une liste peut contenir également plusieurs valeurs de types différents, par exemple pour stocker en une seule variable une série de données :

```
>>> data = ['text', 2.0, 1, [2, 3]]
>>> print(data)
['text', 2.0, 1, [2, 3]]
>>> len(data)
4
```

Comprenez-vous pourquoi `len(data)` retourne 4 et pas 5 ?

Remarque : mélanger les types est assez rare. Nous utiliserons principalement des listes qui contiennent des données du même type.

Accéder et modifier les éléments d'une liste.

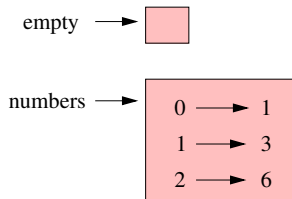
On peut accéder aux éléments d'une liste grâce à l'opérateur `[.]`. Les indices fonctionnent de la même manière que pour les chaînes de caractères.

```
>>> data = ['text', 2.0, 1, [2, 3]]
>>> print(data[0])
text
>>> print(data[-1])
[2, 3]
```

Accéder et modifier les éléments d'une liste.

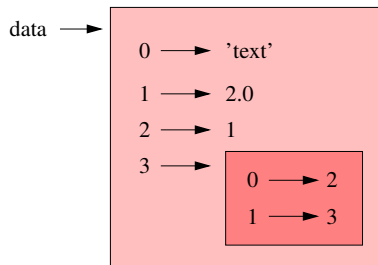
On peut représenter les listes par un diagramme d'état qui illustre les relations entre les indices et les éléments.

```
>>> empty = []  
>>> numbers = [1, 3, 6]
```



Accéder et modifier les éléments d'une liste.

```
>>> data = ['text', 2.0, 1, [2, 3]]
```



Démonstration : représentation de `data` via Python Tutor

Accéder et modifier les éléments d'une liste.

Contrairement aux chaînes qui sont immuables, on peut modifier les éléments d'une liste.

```
>>> mot = 'bonjour'
>>> numbers = [1, 3, 6]
>>> mot[0] = 'B'
```

```
Traceback (most recent call last):
```

```
File "<pyshell#36>", line 1, in <module>
```

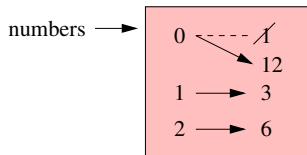
```
    mot[0] = 'B'
```

```
TypeError: 'str' object does not support item assignment
```

```
>>> numbers[0] = 12
```

```
>>> print(numbers)
```

```
[12, 3, 6]
```



Boucles `for`

Une liste est un objet itérable : elle peut donc être utilisée dans une boucle `for`.

```
data = ['text', 2.0, 1, [2, 3]]
```

```
for item in data:  
    print(item, type(item))
```

```
text <class 'str'>  
2.0 <class 'float'>  
1 <class 'int'>  
[2, 3] <class 'list'>
```

Convertir un objet en liste

La fonction `list` permet de convertir un objet en liste, si c'est possible. Cela fonctionne notamment avec les objets de type `str` et de type `range` (voir chap. 6).

```
>>> list(range(4))
[0, 1, 2, 3]
>>> list(range(2, 6))
[2, 3, 4, 5]
>>> s = 'hello'
>>> list(s)
['h', 'e', 'l', 'l', 'o']
```

Listes multidimensionnelles

Liste de listes $\simeq$ tableau multidimensionnel.

Exemple : la matrice

$$A = \begin{pmatrix} 1 & 5 & 3 \\ 4 & 2 & 2 \\ 3 & 7 & 9 \end{pmatrix}$$

peut être représentée par

$$A = [[1, 5, 3], [4, 2, 2], [3, 7, 9]]$$

L'accès à un élément de la $i^{\text{ème}}$ ligne, $j^{\text{ème}}$ colonne se fait via

$$A[i-1][j-1]$$

Par exemple l'élément au centre de la matrice ci-dessus est accédé via

$$A[1][1]$$

Exercices

Exercice

Ecrire une fonction qui retourne la plus grande valeur d'une liste contenant des entiers.

Précondition : on suppose que la liste n'est pas vide.

Exercice

Modifiez le code de l'exercice précédent pour gérer le cas d'une liste vide. Dans ce cas, il faut retourner `None`.

Exercices

Cas avec précondition spécifiant que la liste n'est pas vide :

```
def max_liste(t):  
    max = t[0]  
    for x in t:  
        if x > max:  
            max = x  
    return max
```

Exercices

Cas avec liste vide gérée :

```
def max_liste(t):  
    if len(t) == 0:  
        return None  
    else:  
        max = t[0]  
        for x in t:  
            if x > max:  
                max = x  
        return max
```

Exercices

Variante qui utilise les indices :

```
def max_liste(t):  
    if len(t) == 0:  
        return None  
    else:  
        max = t[0]  
        for i in range(1, len(t)):  
            if t[i] > max:  
                max = t[i]  
        return max
```

Grâce aux indices, on peut éviter de tester le premier élément.

Exercices

Exercice

Ecrire une fonction qui retourne la plus grande valeur d'une matrice $n \times m$ contenant des entiers.

Précondition : on suppose que n et m sont ≥ 1 et que la matrice donnée en entrée est correctement codée (c'est à dire une liste de n listes de m entiers).

Exercices

Variante qui utilise les indices :

```
def max_matrix(A):  
    n = len(A)  
    m = len(A[0])  
    max = A[0][0]  
  
    for i in range(n):  
        for j in range(m):  
            if A[i][j] > max:  
                max = A[i][j]  
    return max  
  
A = [[1, 5, 3], [4, 2, 2], [3, 7, 9], [6, 8, 9]]  
print(max_matrix(A))
```

Exercices

Exercice

Ecrire une fonction qui affiche correctement une matrice $n \times m$ contenant des entiers dans la console.

Précondition : on suppose que n et m sont ≥ 1 et que la matrice donnée en entrée est correctement codée (c'est à dire une liste de n listes de m entiers).

Exercices

```
def print_matrix(A):  
    n = len(A)  
    m = len(A[0])  
  
    for i in range(n):  
        for j in range(m):  
            print(A[i][j], end = ' ' )  
        print('')  
  
A = [[1, 5, 3], [4, 2, 2], [3, 7, 9], [6, 8, 9]]  
print_matrix(A)
```

Listes et opérateurs

Listes et opérateurs

Comme pour les chaînes, l'opérateur + concatène deux listes, l'opérateur * répète une liste un certain nombre de fois.

```
>>> a = [1, 2, 3]
>>> b = [4, 5, 6]
>>> c = a + b
>>> print(c)
[1, 2, 3, 4, 5, 6]
>>> [0] * 4
[0, 0, 0, 0]
>>> a * 3
[1, 2, 3, 1, 2, 3, 1, 2, 3]
```

Opérateur `in`

Comme pour toutes les séquences, l'opérateur `in` est applicable sur une liste.

```
>>> cheeses = ['Cheddar', 'Edam', 'Gouda']
>>> 'Edam' in cheeses
True
>>> 'Brie' in cheeses
False
```

Tranches de listes

L'opérateur `[ : ]` fonctionne aussi sur les listes.

```
>>> t = list(range(10))
>>> print(t)
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
>>> t[1:3]
[1, 2]
>>> t[:4]
[0, 1, 2, 3]
>>> t[3:]
[3, 4, 5, 6, 7, 8, 9]
```

Il peut même être utilisé dans le membre gauche d'une assignation.

```
>>> t[1:3] = [34, 26]
>>> print(t)
[0, 34, 26, 3, 4, 5, 6, 7, 8, 9]
```

Copie d'une liste

Comme les listes sont mutables, il est parfois utile de faire une copie avant de les modifier. On peut créer une copie avec une “tranche complète”.

```
>>> copy = t[:]
>>> t[4:6] = [0] * 2
>>> print(t, copy)
[0, 34, 26, 3, 0, 0, 6, 7, 8, 9] [0, 34, 26, 3, 4, 5, 6, 7, 8, 9]
```

Remarque : comme on va le voir à la fin de ce chapitre, cette méthode ne fonctionne correctement que si la liste ne contient que des objets immuables.

Méthodes sur les listes

Méthodes sur les listes

Les objets de type `list` possèdent une série de méthodes utiles.

```
>>> dir(list)
[...
'append', 'clear', 'copy', 'count', 'extend', 'index', 'insert', 'pop',
'remove', 'reverse', 'sort']
```

Méthodes sur les listes

La méthode `append` ajoute un nouvel élément à la fin de la liste.

```
>>> help(list.append)
Help on method_descriptor:
```

```
append(...)
    L.append(object) -> None -- append object to end
```

```
>>> t = ['a', 'b', 'c']
>>> t.append('d')
>>> print(t)
['a', 'b', 'c', 'd']
```

Méthodes sur les listes

La méthode `pop` **supprime et retourne** le dernier élément de la liste.

```
>>> help(list.pop)
Help on method_descriptor:

pop(...)
    L.pop([index]) -> item -- remove and return item at index (default last).
    Raises IndexError if list is empty or index is out of range.

>>> t = ['a', 'b', 'c', 'd']
>>> deleted = t.pop()
>>> print(deleted)
d
>>> print(t)
['a', 'b', 'c']
```

Si l'argument optionnel est défini, supprime et retourne l'élément situé à l'indice donné.

```
>>> t.pop(1)
'b'
>>> print(t)
['a', 'c']
```

Méthodes sur les listes

La méthode `sort` trie les éléments d'une liste. Ses arguments sont optionnels : par exemple, `reverse` permet de trier par ordre décroissant.

```
>>> help(list.sort)
Help on method_descriptor:

sort(...)
    L.sort(key=None, reverse=False) -> None -- stable sort *IN PLACE*

>>> t = ['d', 'c', 'e', 'b', 'a']
>>> t.sort()
>>> print(t)
['a', 'b', 'c', 'd', 'e']
>>> t.sort(reverse=True)
>>> print(t)
['e', 'd', 'c', 'b', 'a']
```

D'autres méthodes utiles sont présentées dans la section "Pour aller plus loin" à lire par vous même, à la fin de ce chapitre.

Exercice

Exercice

Ecrire un script qui demande à l'utilisateur le nom d'un fichier de texte (`.txt`), affiche les mots contenus dans le fichier par ordre alphabétique (en minuscules). Un même mot ne peut pas être affiché deux fois. Tous les caractères qui ne sont pas des lettres ou des chiffres seront supprimés dans la création de la liste de mots.

Hints :

- la méthode `isalnum` invoquée sur une chaîne retourne vrai ssi tous les caractères de la chaîne sont des lettres ou des chiffres.
- la méthode `split` invoquée sur une chaîne retourne une liste des “mots” de la chaîne (c-à-d les sous-chaînes séparées par des caractères blancs¹). On peut spécifier d'autres séparateurs que des blancs avec le paramètre optionnel `sep`.

1. Espaces, retours à la ligne, tabulations, etc.

Exercice

```
>>> help(str.split)
```

```
Help on method_descriptor:
```

```
split(...)
```

```
    S.split(sep=None, maxsplit=-1) -> list of strings
```

Return a list of the words in S, using sep as the delimiter string. If maxsplit is given, at most maxsplit splits are done. If sep is not specified or is None, any whitespace string is a separator and empty strings are removed from the result.

```
>>> 'Un deux trois'.split()
```

```
['Un', 'deux', 'trois']
```

```
>>> '000-123456-47'.split('-')
```

```
['000', '123456', '47']
```

```
>>> '000-123456-47'.split('-', 1)
```

```
['000', '123456-47']
```

Exercice

On définit une fonction `clean` qui retourne la chaîne passée en argument en minuscules, et en lui ayant retiré tous les caractères qui ne sont pas des lettres ou des chiffres.

```
def clean(s):  
    res = ''  
    for letter in s:  
        if letter.isalnum():  
            res = res + letter  
    return res.lower()
```

```
>>> clean('He#23.?6Xth')  
he236xth
```

Exercice

```
filename = input('Nom du fichier: ')
with open(filename) as file:
    wordsList = []
    for line in file:
        for word in line.split():
            cleanWord = clean(word)
            if not cleanWord in wordsList:
                wordsList.append(cleanWord)

wordsList.sort()
print(wordsList)
```

Objets et valeurs

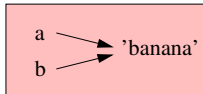
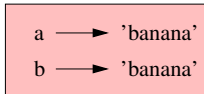
Objets et valeurs

Si on exécute les assignments suivantes, on sait que `a` et `b` réfèrent à une chaîne, mais on ne sait pas si ces variables réfèrent la *même* chaîne.

```
>>> a = 'banana'
```

```
>>> b = 'banana'
```

Il y a deux possibilités :



Dans un cas, `a` et `b` réfèrent deux différents objets qui ont la même valeur. Dans l'autre, elles réfèrent au même objet.

Objets et valeurs

```
>>> a = 'banana'  
>>> b = 'banana'
```

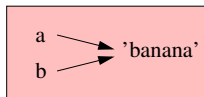
L'opérateur `is` permet de tester si deux variables réfèrent le même objet.

Pensez-vous que `a is b` ?

Objets et valeurs

```
>>> a = 'banana'  
>>> b = 'banana'  
>>> a is b  
True
```

Donc, dans ce cas, Python crée un seul objet “chaîne” et les deux variables réfère à cet objet.



Objets et valeurs

Qu'en est-il des listes ?

```
>>> a = [1, 2, 3]
```

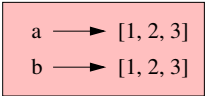
```
>>> b = [1, 2, 3]
```

Pensez-vous que `a is b` ?

Objets et valeurs

Dans ce cas, Python crée deux objets différents.

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> a is b
False
```



a → [1, 2, 3]
b → [1, 2, 3]

Remarques :

- les deux listes sont **équivalentes**, car elles ont les mêmes valeurs, mais elles ne sont pas **identiques**, car elles ne sont pas le même objet ;
- si deux objets sont identiques, ils sont aussi équivalents, mais le contraire n'est pas toujours vrai.

Objets et valeurs

Pourquoi deux comportements différents en créant deux chaînes ou deux listes ?

```
>>> 'banana' is 'banana'
```

```
True
```

```
>>> [1, 2, 3] is [1, 2, 3]
```

```
False
```

Objets et valeurs

Pourquoi deux comportements différents en créant deux chaînes ou deux listes ?

```
>>> 'banana' is 'banana'
True
>>> [1, 2, 3] is [1, 2, 3]
False
```

Car les chaînes sont immuables et les listes mutables.

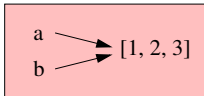
- Si on modifie par après une des deux listes, l'autre n'est pas modifiée : elles ont “pour le moment” la même valeur, mais elles peuvent évoluer de manière indépendante, puisqu'elles sont des objets différents.
- Pour les objets immuables, on peut se permettre d'avoir un seul objet, ce qui économise de l'espace mémoire.
- C'est donc la mutabilité d'un objet qui détermine ce comportement.

Alias

Dans certains cas, on veut que deux variables réfèrent le même objet même s'il est mutable.

On appelle cela un **alias** : il suffit d'assigner la première variable à la seconde (plutôt qu'une valeur équivalente).

```
>>> a = [1, 2, 3]
>>> b = a
>>> a is b
True
```



Alias

Modifier un objet qui est référencé par plusieurs variables est visible depuis l'ensemble de celles-ci.

```
>>> a = [1, 2, 3]
>>> b = [1, 2, 3]
>>> c = b
>>> print(a, b, c)
[1, 2, 3] [1, 2, 3] [1, 2, 3]
>>> a is b, a is c, b is c
(False, False, True)
>>> a[0] = 4
>>> print(a, b, c)
[4, 2, 3] [1, 2, 3] [1, 2, 3]
>>> b[0] = 5
>>> print(a, b, c)
[4, 2, 3] [5, 2, 3] [5, 2, 3]
```

Exercice : utilisez Python Tutor pour explorer comment sont représentés `a`, `b` et `c`

Objets mutables en arguments

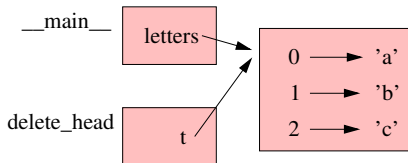
Quand on passe une liste en argument d'une fonction, la fonction reçoit une **référence** vers la liste : le paramètre de la liste est un alias et les modifications de la liste sont visibles en dehors de la fonction.

```
>>> def delete_head(t):  
    t.pop(0)  
  
>>> letters = ['a', 'b', 'c']  
>>> delete_head(letters)  
>>> print(letters)  
['b', 'c']
```

Objets mutables en arguments

Quand on passe une liste en argument d'une fonction, la fonction reçoit une **référence** vers la liste : le paramètre de la liste est un alias et les modifications de la liste sont visibles en dehors de la fonction.

```
>>> def delete_head(t):  
    t.pop(0)  
  
>>> letters = ['a', 'b', 'c']  
>>> delete_head(letters)  
>>> print(letters)  
['b', 'c']
```



(voir code `delete_head` dans Python Tutor)

La fonction ci-dessus a-t-elle un effet de bord ?

Objets mutables en arguments

Rappel : cf. Chapitre 3.

Définition

On dit qu'une fonction possède un **effet de bord** si celle-ci produit un effet qui est visible en dehors de la fonction. Le fait de retourner une valeur n'est pas considéré comme un effet de bord.

Objets mutables en arguments

Rappel : cf. Chapitre 3.

Définition

On dit qu'une fonction possède un **effet de bord** si celle-ci produit un effet qui est visible en dehors de la fonction. Le fait de retourner une valeur n'est pas considéré comme un effet de bord.

Donc, oui, une fonction qui modifie un objet mutable passé en argument possède un effet de bord.

Comment écrire une fonction qui ferait le même travail mais sans effet de bord ?

Objets mutables en arguments

Rappel : cf. Chapitre 3.

Définition

On dit qu'une fonction possède un **effet de bord** si celle-ci produit un effet qui est visible en dehors de la fonction. Le fait de retourner une valeur n'est pas considéré comme un effet de bord.

Donc, oui, une fonction qui modifie un objet mutable passé en argument possède un effet de bord.

Comment écrire une fonction qui ferait le même travail mais sans effet de bord ?

Puisqu'on ne peut pas modifier les paramètres, il faut **retourner** un nouvel objet (ici une nouvelle liste).

Objets mutables en arguments

Voici une version sans effet de bord qui exploite le fait que l'opérateur `[ : ]` retourne une nouvelle liste.

```
>>> def safe_del_head(t):  
    return t[1:]  
  
>>> letters = ['a', 'b', 'c']  
>>> new_letters = safe_del_head(letters)  
>>> print(letters)  
['a', 'b', 'c']  
>>> print(new_letters)  
['b', 'c']
```

(voir code `safe_delete_head` dans Python Tutor)

Objets mutables en arguments

Une fonction sans effet de bord peut aussi être utilisée pour mettre à jour une liste passée en paramètres, grâce à l'assignation.

```
>>> letters = ['a', 'b', 'c']  
>>> letters = safe_del_head(letters)  
>>> print(letters)  
['b', 'c']
```

Elle offre donc le **contrôle** de choisir parmi les deux possibilités, contrairement à une fonction avec effet de bord.

Objets mutables en arguments

Il est important de faire la distinction entre ce qui modifie un objet et ce qui en retourne un nouveau.

Par exemple, la méthode `append` modifie la liste (effet de bord) (et retourne `None`).

```
>>> t1 = [1, 2]
>>> t2 = t1.append(3)
>>> print(t1, t2)
[1, 2, 3] None
```

Par contre, l'opérateur `+` retourne une nouvelle liste.

```
>>> t3 = t1 + [3]
>>> print(t1, t3)
[1, 2, 3] [1, 2, 3, 3]
```

Objets mutables en arguments

En général :

- les **fonctions** sont **sans effet de bord** (par ex. `math.sqrt`)
- les **méthodes** sont
 - ▶ **avec un effet de bord** si l'objet est **mutable** (par ex. `list.append`, `list.pop`, `list.sort`)
 - ▶ **sans effet de bord** si l'objet est **immuable** (par ex. `str.upper`, `str.lower`)
- les **opérateurs** sont **sans effet de bord** (par ex. `+` et `[ : ]` pour les chaînes et les listes)

Exercice

Pourquoi la fonction suivante, censée supprimer le premier élément de la liste passée en paramètre, ne fonctionne pas ?

```
>>> def bad_delete_head(t):  
    t = t[1:]  
  
>>> letters = ['a', 'b', 'c']  
>>> bad_delete_head(letters)  
>>> print(letters)  
['a', 'b', 'c']
```

(voir code `bad_delete_head` dans Python Tutor)

Exercice

Pourquoi la fonction suivante, censée supprimer le premier élément de la liste passée en paramètre, ne fonctionne pas ?

```
>>> def bad_delete_head(t):  
    t = t[1:]  
  
>>> letters = ['a', 'b', 'c']  
>>> bad_delete_head(letters)  
>>> print(letters)  
['a', 'b', 'c']
```

- l'opérateur `[ : ]` crée une nouvelle liste et `t` réfère maintenant cette nouvelle liste
- le fait d'avoir changé la référence de `t` (intuition : la flèche sur les diagrammes d'état), ne signifie pas qu'on a touché à celle de `letters` (qui réfère toujours la liste de départ)

(voir code `bad_delete_head` dans Python Tutor)

Liste de listes et copies

Avec des listes de listes, il y a quelques subtilités liées à la copie.

Que va afficher la dernière instruction ?

```
>>> A = [[0] * 3] * 2
>>> print(A)
[[0, 0, 0], [0, 0, 0]]
>>> A[0][0] = 1
>>> print(A)
```

Liste de listes et copies

Avec des listes de listes, il y a quelques subtilités liées à la copie.

Que va afficher la dernière instruction ?

```
>>> A = [[0] * 3] * 2
>>> print(A)
[[0, 0, 0], [0, 0, 0]]
>>> A[0][0] = 1
>>> print(A)
```

On aimerait (et on s'attend à) obtenir

```
[[1, 0, 0], [0, 0, 0]]
```

Liste de listes et copies

Avec des listes de listes, il y a quelques subtilités liées à la copie.

Que va afficher la dernière instruction ?

```
>>> A = [[0] * 3] * 2
>>> print(A)
[[0, 0, 0], [0, 0, 0]]
>>> A[0][0] = 1
>>> print(A)
```

On aimerait (et on s'attend à) obtenir

```
[[1, 0, 0], [0, 0, 0]]
```

mais on obtient

```
[[1, 0, 0], [1, 0, 0]]
```

Les deux “lignes” de cette matrice à 2 dimensions sont donc des alias
→ ce n'était sûrement pas ce qui était souhaité !

Liste de listes et copies

Que va afficher ceci ?

```
>>> A = [[1, 2], [3, 4]]  
>>> B = A[:]  
>>> A[0][0] = 9  
>>> print(A, B)
```

Liste de listes et copies

Que va afficher ceci ?

```
>>> A = [[1, 2], [3, 4]]
>>> B = A[:]
>>> A[0][0] = 9
>>> print(A, B)
```

On s'attend à ce que B soit une copie distincte de A comme ce serait le cas si A était une simple liste d'objets immuables :

```
[[9, 2], [3, 4]] [[1, 2], [3, 4]]
```

Liste de listes et copies

Que va afficher ceci ?

```
>>> A = [[1, 2], [3, 4]]
>>> B = A[:]
>>> A[0][0] = 9
>>> print(A, B)
```

On s'attend à ce que B soit une copie distincte de A comme ce serait le cas si A était une simple liste d'objets immuables :

```
[[9, 2], [3, 4]] [[1, 2], [3, 4]]
```

mais on obtient :

```
[[9, 2], [3, 4]] [[9, 2], [3, 4]]
```

Liste de listes et copies

Explication : les 2 situations précédentes sont dues au fait que l'on copie les **références** vers les objets mutables que sont les sous-listes. Les opérateurs `*` et `[:]` ne réalisent pas une copie “en profondeur” : ils copient les références mais ne les suivent pas.

(Voir codes `matrix` dans Python Tutor)

Solution ? Utiliser la fonction `deepcopy` du module `copy`. Celle-ci va “suivre” les références pour effectuer une copie réellement distincte (quel que soit le nombre ou la profondeur des références à suivre).

```
>>> import copy
>>> A = [[1, 2], [3, 4]]
>>> B = copy.deepcopy(A)
>>> A[0][0] = 9
>>> print(A, B)
[[9, 2], [3, 4]] [[1, 2], [3, 4]]
```

Arguments de la ligne de commande (parenthèse)

Arguments de la ligne de commande

La fonction `input` permet d'obtenir des entrées de l'utilisateur.

Un autre moyen (rapide et donc souvent apprécié à l'utilisation) d'obtenir des entrées de l'utilisateur, est de récupérer les “arguments de la ligne de commande”.

Exemples :

- la commande `cd myDir` vous permet de changer de répertoire dans une console. Le nom du répertoire (`myDir`) est un argument de la commande `cd`.
- la commande `python3 myscript.py` vous permet d'interpréter le script `myscript.py`. Le nom du script est un argument de la commande `python3` qui lance l'interpréteur.

Arguments de la ligne de commande

Tous les arguments passés après la commande `python3` (et séparés par des espaces²) sont disponibles via l'attribut `argv` du module `sys`. La valeur `argv` est une liste qui contient ces arguments.

Fichier `args.py` :

```
import sys
print(sys.argv)
```

La commande `"python3 args.py"` produit :

```
['args.py']
```

La commande `"python3 args.py un deux trois 'x + y'"` produit :

```
['args.py', 'un', 'deux', 'trois', 'x + y']
```

2. Pour passer un argument qui contient des espaces, on peut le mettre entre apostrophes.

Arguments de la ligne de commande

Amélioration du script qui trie les mots d'un fichier :

...

```
import sys
```

```
filename = ''
```

```
if len(sys.argv) < 2:
```

```
    filename = input('Nom du fichier: ')
```

```
else:
```

```
    filename = sys.argv[1]
```

```
with open(filename) as file:
```

...

Erreurs fréquentes et debug

Erreurs fréquentes et debug

Une utilisation non rigoureuse des listes (ou d'autres objets mutables) peut mener à des problèmes difficiles à déboguer.

- la plupart des méthodes sur les listes modifient celles-ci (et retournent `None`). C'est l'inverse pour les méthodes sur les chaînes (qui ne pourraient pas modifier l'argument) : elles retournent une nouvelle chaîne.

Exemple :

```
word = word.strip()  
t = t.sort()    # FAUX: t refere None !
```

- il faut donc lire avec attention la documentation des fonctions, méthodes et opérateurs ; et écrire votre propre documentation de manière claire. Le mode interactif est très utile pour tester rapidement l'utilisation d'une fonction ou d'une méthode.

Erreurs fréquentes et debug

- faites une copie si nécessaire.

```
import copy
t_original = copy.deepcopy(t)
t.sort()
```

- *Choisissez une manière de faire, apprenez-la correctement.*

Avec les listes $\exists$ de nombreuses manières de faire la même chose :

- ▶ pour supprimer un élément d'une liste : `pop`, `remove` ou même `[x:y]`.
- ▶ pour ajouter un élément d'une liste : `append` ou opérateur `+`

Erreurs fréquentes et debug

Ces instructions sont correctes pour ajouter un élément :

```
t.append(x)  
t = t + [x]
```

Ces instructions sont erronées :

```
t.append([x])  
t = t.append(x)  
t + [x]  
t = t + x
```

Exercice : essayez ces exemples interactivement pour comprendre ce qu'ils font (seule la dernière instruction provoque une exception).

Exercices interactifs

Exercices interactifs

Exercice

Ecrire un programme qui affiche les paires de mots (contenus dans le fichier `words.txt`) qui sont l'inverse l'un de l'autre. Par exemple, `stop` et `pots`.

Exercice

Ecrire un programme qui estime la probabilité que $k \geq 2$ étudiants aient leur anniversaire le même jour au sein d'une classe de n étudiants. Les paramètres n et k sont passés en ligne de commande.

Illustrations

Présentation et illustration des codes de :

- la poésie aléatoire
- l'énigme de la traversée
- la solution complète des tortues dans des labyrinthes

Pour aller plus loin (à lire par soi-même)

A propos des listes dans une boucle `for`

Pour accéder aux indices et aux valeurs d'une liste, on peut utiliser `enumerate` :

```
>>> t = [3, 5, 2, 99]
>>> for i, x in enumerate(t):
    print(i, x)
```

```
0 3
1 5
2 2
3 99
```

Une boucle `for` sur une liste vide n'exécute jamais son corps.

```
for x in []:
    print('This never happens.')
```

Exemple d'utilisation des opérateurs sur les listes

Comment construire la liste suivante ?

`[1,3,0,0,9,11,13,15,1,3,0,0,9,11,13,15]`

```
>>> liste = list(range(1, 16, 2))
>>> print(liste)
[1, 3, 5, 7, 9, 11, 13, 15]
>>> liste[2:4] = [0] * 2
>>> print(liste)
[1, 3, 0, 0, 9, 11, 13, 15]
>>> liste = liste * 2
>>> print(liste)
[1, 3, 0, 0, 9, 11, 13, 15, 1, 3, 0, 0, 9, 11, 13, 15]
```

D'autres méthodes utiles sur les listes

La méthode `extend` prend une liste en argument et ajoute tous ses éléments en fin de la liste.

```
>>> help(list.extend)
Help on method_descriptor:

extend(...)
    L.extend(iterable) -> None -- extend list by appending elements
                                from the iterable

>>> t1 = ['a', 'b', 'c']
>>> t2 = ['d', 'e']
>>> t1.extend(t2)
>>> print(t1)
['a', 'b', 'c', 'd', 'e']
```

D'autres méthodes utiles sur les listes

La méthode `count` compte toutes les occurrences d'une valeur passée en argument.

```
>>> help(list.count)
Help on method_descriptor:

count(...)
    L.count(value) -> integer -- return number of occurrences of value

>>> t = ['a', 'b', 'c', 'a']
>>> t.count('a')
2
```

D'autres méthodes utiles sur les listes

La méthode `insert` permet d'insérer un élément à un endroit donné de la liste.

```
>>> help(list.insert)
Help on method_descriptor:
```

```
insert(...)
    L.insert(index, object) -- insert object before index
```

```
>>> t = ['a', 'b', 'c']
>>> t.insert(0, 'z')
>>> print(t)
['z', 'a', 'b', 'c']
>>> t.insert(2, 'x')
>>> print(t)
['z', 'a', 'x', 'b', 'c']
```

D'autres méthodes utiles sur les listes

La méthode `remove` supprime la première occurrence d'une valeur donnée.

```
>>> help(list.remove)
Help on method_descriptor:
```

```
remove(...)
    L.remove(value) -> None -- remove first occurrence of value.
    Raises ValueError if the value is not present.
```

```
>>> t = ['a', 'b', 'c', 'b']
>>> t.remove('b')
>>> print(t)
['a', 'c', 'b']
>>> t.remove('z')
```

```
Traceback (most recent call last):
```

```
File "<pyshell#100>", line 1, in <module>
    t.remove('z')
```

```
ValueError: list.remove(x): x not in list
```

D'autres méthodes utiles sur les listes

La méthode `index` retourne l'indice de la première occurrence d'une valeur donnée (des arguments optionnels permettent de limiter la recherche à un intervalle d'indices).

```
>>> help(list.index)
Help on method_descriptor:
```

```
index(...)
    L.index(value, [start, [stop]]) -> integer -- return first index of value.
    Raises ValueError if the value is not present.
```

```
>>> t = ['a', 'b', 'c', 'a']
>>> t.index('a')
0
>>> t.index('z')
```

```
Traceback (most recent call last):
  File "<pyshell#78>", line 1, in <module>
    t.index('z')
ValueError: list.index(x): x not in list
>>> t.index('a',1)
3
```

Supprimer les éléments d'une liste

Outre les méthodes `remove` et `pop`, l'opérateur `del` peut également être utilisé pour supprimer un élément à partir d'un indice (ou d'une tranche d'indices).

```
>>> t = ['a', 'b', 'c', 'd']
>>> x = t.pop(1)
>>> print(x)
b
>>> print(t)
['a', 'c', 'd']
>>> x = t.remove('c')
>>> print(x)
None
>>> print(t)
['a', 'd']
>>> del t[1]
>>> print(t)
['a']
>>> t = ['a', 'b', 'c', 'd', 'e', 'f']
>>> del t[1:5]
>>> print(t)
['a', 'f']
```

Listes et chaînes

Un `str` est une séquence de caractères et une liste est une séquence de valeurs, mais une liste de caractères n'est pas la même chose qu'une chaîne.

Pour convertir une chaîne en liste, on peut utiliser `list`.

```
>>> s = 'hello'
>>> t = list(s)
>>> print(t)
['h', 'e', 'l', 'l', 'o']
```

- Comme `list` est le nom d'une fonction, on ne peut pas l'utiliser comme nom de variable (mais `liste` peut l'être).
- En général, on évite d'utiliser la lettre `l` car elle ressemble trop au nombre 1, c'est pourquoi `t` est souvent utilisé dans les exemples.

Listes et chaînes

La fonction `list` sépare une chaîne en caractères individuellement. Nous avons vu la méthode `split` qui permet de séparer une chaîne en mots ou en sous-chaînes.

La méthode `join` fait le contraire de `split`. C'est une méthode sur les chaînes : on l'invoque sur le délimiteur et on passe la liste en paramètre (elle peut être utile après un `map`).

```
>>> s = 'to be or not to be'
>>> t = s.split()
>>> print(t)
['to', 'be', 'or', 'not', 'to', 'be']
>>> s = 'spam*spam*spam'
>>> delimiter = '*'
>>> s.split(delimiter)
['spam', 'spam', 'spam']
>>> delimiter = ' '
>>> delimiter.join(t)
'to be or not to be'
>>> ''.join(t)
'tobeornottobe'
```

Glossaire

Glossaire

élément : une des valeurs dans une liste (ou une autre séquence).

indice : valeur entière qui indique la position d'un élément dans une liste (ou une séquence).

liste : séquence de valeurs qui peuvent être modifiées et mises entre crochets droits.

liste imbriquée : liste qui est un élément d'une autre liste.

accumulateur : variable utilisée dans une boucle pour additionner ou accumuler un résultat.

filtre : opération qui consiste à traverser une liste (ou une séquence) et qui sélectionne les éléments satisfaisant un certain critère.

mapping : opération qui consiste à traverser une liste (ou une séquence) pour appliquer une action sur chaque élément.

réduction : opération qui consiste à traverser une liste (ou une séquence) et qui accumule les éléments en un seul résultat.

object : quelque chose qu'une variable peut référer. Un objet possède un type et une valeur. On peut lui appliquer des méthodes définies en fonction de son type.

référence : l'association entre une variable et sa valeur (c-à-d la valeur de son objet).

alias : assigner une variable depuis une autre, pour qu'elles réfèrent au même objet.

équivalent : ayant la même valeur.

identique : être le même objet (ce qui implique l'équivalence).